

role	repo + file	workload	workload evidence	thread model	thread model evidence	primitives	tradeoff notes
Logging system	alacrity/alacrity:alacrity/src/logger	I/O-bound	"all threads write to the same logfile", "Multiple threads can log concurrently - > writes should be serialised while notifying UI", "stdout: Mutex<LineWriter<Stdout>> -> same thing as before, needs to be serialised", "Formatting happens before locks -> only actual writes serialised, so it doesn't hold lock for longer than it needs"	Shared-state	"Logger: logfile: Mutex<OnDemandLogFile>, stdout: Mutex<LineWriter<Stdout>>, event_proxy: Mutex<EventLoopProxy<Event>>", "Any thread can call log() concurrently", "logging is serialised with mutexes to maintain consistency of output", "all threads write to the same logfile, this ensures only one thread at a time does that, in log(): acquire lock-write-release lock"	Mutex, Arc<AtomicBool>	Formatting happens before locks so only actual writes are serialised, reducing lock hold time while
		Coordination	"Prevent duplicate loading and avoid repeatedly querying OpenGL data", "Lock-free atomic coordination, single initialisation", "Multiple renderer initialisation threads", "OpenGL initialisation and cache extension capabilities across renderer threads", "Not much shared memory here, but it is mostly not necessary, because only one thread mutates a renderer at a time"	concurrency model	"GL_FUNS_LOADED: AtomicBool = AtomicBool::new(false)", "only one thread will see that it is false and initialise", "static OPENG_L_EXTENSIONS: OnceLock<HashSet<&static	OnceLock	still protecting shared I/O state
Rendering coordination	alacrity/alacrity:alacrity/src/renderer	n-heavy	"The main event loop which performs I/O on the pseudoterminal and rendering coordination", "The PTY thread polls the PTY for readable/writable events, reads shell output, parses escape sequences, updates shared terminal state", "PTY event-loop thread, UI/render thread, input/notifier threads", "thread updates terminal from shell output while UI/render thread reads and renders safely without starvation"	Initialisation / singleton str, RandomState>>, "initialise-once (get_or_init)"	"Arc<FairMutex<TermCU>>> -> shared terminal", "PTY thread acquires fair lease, attempts try_lock_unfair, falls back to blocking lock", "self.terminal.lock_unfair()",	AtomicBool, OnceLock<T>	avoiding runtime locking costs
		Mixed	"The entire file is about controlling shared-memory access fairness, prevent starvation"	Resource arbitration	"pub struct FairMutex<T> with data: Mutex<T>, next: Mutex<()> -> data protects the real shared memory", "before it goes to the data, every thread must first acquire the next", "Split between reservation (next) and ownership of data"	Arc<FairMutex<T>>, Arc<T>	Fair locking prevents renderer starvation during heavy PTY throughput, but introduces additional
I/O coordination	alacrity/alacrity:alacrity_terminal	Coordination	"The entire file is about controlling shared-memory access fairness, prevent starvation"	model	"MAX_LOCKED_READ = for preventing PTY thread monopolising terminal lock forever"		synchronisation and coordination overhead
Synchronisation primitives	alacrity/alacrity:alacrity_terminal	n-heavy	"WinAPI must safely communicate process exit back to event loop threads"	Resource arbitration	"pub struct FairMutex<T> with data: Mutex<T>, next: Mutex<()> -> data protects the real shared memory", "before it goes to the data, every thread must first acquire the next", "Split between reservation (next) and ownership of data"	parking_lot::Mutex<T>	Provides strong fairness guarantees and starvation prevention, but every lock acquisition requires
		Coordination	"Async favicon fetching, parsing, caching, sanitisation, and delivery service", "Global HTTP client, cookie store, compiled regex, pooled sockets/TLS sessions", "Tokio async runtime with many concurrent request tasks sharing state", "static CLIENT: LazyLock<Client>", "use arc to clone references to the cookie store across threads"	Actor / message-	"Callback thread atomically reads process handle, sends exit event through channel", "Windows wait-thread callback + application event loop/poller thread", "AtomicPtr<c_void> used to share child process handle safely across threads/callback boundary"	Arc<T>, Arc<Mutex<Option<T>>>, AtomicPtr<T>	layered coordination through an additional mutex
OS integration bridge	alacrity/alacrity:alacrity_terminal	n-heavy	"Async favicon fetching, parsing, caching, sanitisation, and delivery service", "Global HTTP client, cookie store, compiled regex, pooled sockets/TLS sessions", "Tokio async runtime with many concurrent request tasks sharing state", "static CLIENT: LazyLock<Client>", "use arc to clone references to the cookie store across threads"	passing model	"Tokio async runtime with many concurrent request tasks sharing state", "Async favicon fetching, parsing, caching, sanitisation, and delivery service", "Mostly read-heavy shared access", "Arc::clone(&cookie_store)", "RwLock<CookieStore>"	LazyLock, Arc, RwLock	Avoids blocking inside WinAPI callbacks through atomics while limiting mutex use to mutable registration state
Caching layer	dani-garcia/vaultwarden:src/api/ico	I/O-bound	"Centralise the system configuration, avoid recomputation, share IO clients, allow reconfiguration at runtime", "Config: LazyLock<Config> -> singleton, and then shared everywhere. inner: RwLock<Inner> -> high read, rare writes (for startup / reload).", "update_config, delete_user_config, set_rocket_shutdown_handle: write heavy, takes write lock and reassigns whole inner.", "pub static SKIP_CONFIG_VALIDATION: AtomicBool -> behavior switch, runtime control -> lock-free shared memory coordination",	coordination	"RwLock<T>", "configuration state (env, usr, overrides, config)", "inner: RwLock<Inner> -> high read, rare writes (for startup / reload)", "takes write lock and reassigns whole inner", "AtomicBool -> behavior switch, runtime control -> lock-free shared memory"	AtomicBool, RwLock<T>	cookie state still requires locking on writes
Shared configuration	dani-garcia/vaultwarden:src/config	n-heavy	"Shared database connection pooling and execution coordination", "Allow many Rocket requests (async) to share mutable Diesel DB connections without blocking the Tokio executor", "Acquire semaphore + pooled connection -> async mutex lock -> execute blocking query -> release connection + permit during Drop", "Semaphore control + async mutual exclusion + pooled synchronisation + initialise once"	Shared-state	"Acquire semaphore + pooled connection -> async mutex lock -> execute blocking query -> release connection + permit during Drop", "DB pool state", "Allow many Rocket requests (async) to share mutable Diesel DB connections", "mutex serialises &mut	LazyLock<T>	Read-heavy configuration access is fast and scalable, but replacing whole snapshots makes updates
		Coordination	"shared HTTP/DNS infrastructure", "HTTP client, DNS resolver, compiled regex"	concurrency model	coordination"	Arc<T>	coarse-grained
Database coordination	dani-garcia/vaultwarden:src/db/mo	n-heavy	"(HTTP_REQUEST_BLOCK_REGEX), shared connection pools, resolver state", "Application state, thread coordination", "Multiple async tasks, Rocket handlers, background jobs and signal handlers must access the same state", "Shared DB pool, websocket registries, configuration state, shutdown handles, runtime access, scheduler resources", "Arc::clone() into async tasks and scheduler closures", "config::SKIP_CONFIG_VALIDATION.store(true, Ordering::Relaxed)"	Resource arbitration	"Acquire semaphore + pooled connection -> async mutex lock -> execute blocking query -> release connection + permit during Drop", "DB pool state", "Allow many Rocket requests (async) to share mutable Diesel DB connections", "mutex serialises &mut	OwnedSemaphorePermit, Arc<Mutex<Option<T>>>,	Blocking Diesel work is isolated from the async runtime, but semaphore and mutex coordination
Resource coordination	dani-garcia/vaultwarden:src/http_d	n-heavy	"Async request/response coordination between debugger transport tasks", "match server responses to originating client request", "Many async tasks read/write the request map, send task inserts pending requests, recv task removes/completes them", "Mutex protects correlation table, while channels carry actual data", "pending_requests: Mutex<HashMap<u64, Sender<Result<Response>>>> -> for each request, a channel is stored so that its response can be delivered later"	model	Connection access", "Arc<Semaphore>"	OnceLock<DbConnType>	can create contention under heavy DB load
		I/O-bound	"shared HTTP/DNS infrastructure", "HTTP client, DNS resolver, compiled regex"	Initialisation / singleton coordination	"LazyLock<T>, LazyLock<Arc<T>>>", "avoid rebuilding clients/resolvers/regexes"	LazyLock<T>, LazyLock<Arc<T>>, Arc<T>, Mutex<Option<String, Regex>>>	Shared clients and resolvers reduce expensive reconstruction overhead, but the regex cache
Runtime coordination	dani-garcia/vaultwarden:src/main.rs	n-heavy	"Coordinate editor redraw timing and prevent rendering during async work", "lets background tasks block frame rendering until computation completes, while also waking renderer efficiently", "RENDER_LOCK: RwLock<()> -> controls when rendering is allowed", "async tasks take the read lock, renderer takes write lock, rendering must wait until all async tasks have completed"	model	("=expensive)", "shared singleton to minimise cost", "shared http client, threads reuse it"	Regex>>>	introduces a small global contention point
		Coordination	"Application state, thread coordination", "Multiple async tasks, Rocket handlers, background jobs and signal handlers must access the same state", "Shared DB pool, websocket registries, configuration state, shutdown handles, runtime access, scheduler resources", "Arc::clone() into async tasks and scheduler closures", "config::SKIP_CONFIG_VALIDATION.store(true, Ordering::Relaxed)"	Async executor-driven	"Multiple async tasks, Rocket handlers, background jobs and signal handlers must access the same state", "Tokio cooperative async scheduling", "runtime.spawn(... pool.clone())", "Rocket worker threads, scheduler thread, spawned Tokio tasks, Unix signal listener tasks", "Arc, atomic synchronisation for flags"	Arc	Arc-based ownership sharing avoids excessive locking, but coordination responsibilities are distributed across runtime tasks and framework boundaries
Request coordination	helix-editor/helix:helix-dap/src/transport	n-heavy	"Async request/response coordination between debugger transport tasks", "match server responses to originating client request", "Many async tasks read/write the request map, send task inserts pending requests, recv task removes/completes them", "Mutex protects correlation table, while channels carry actual data", "pending_requests: Mutex<HashMap<u64, Sender<Result<Response>>>> -> for each request, a channel is stored so that its response can be delivered later"	Coordination	"Multiple async tasks, Rocket handlers, background jobs and signal handlers must access the same state", "Tokio cooperative async scheduling", "runtime.spawn(... pool.clone())", "Rocket worker threads, scheduler thread, spawned Tokio tasks, Unix signal listener tasks", "Arc, atomic synchronisation for flags"	Arc, tokio::sync::Mutex	Mutex protects correlation state safely across async tasks but the shared pending-request table can become a contention hotspot under heavy debugger traffic
		Coordination	"Coordinate editor redraw timing and prevent rendering during async work", "lets background tasks block frame rendering until computation completes, while also waking renderer efficiently", "RENDER_LOCK: RwLock<()> -> controls when rendering is allowed", "async tasks take the read lock, renderer takes write lock, rendering must wait until all async tasks have completed"	Shared-state	"static RENDER_LOCK: RwLock<()> -> controls when rendering is allowed", "async tasks take the read lock, renderer takes write lock", "rendering must wait until all async tasks have completed"	parking_lot::RwLock	Guarantees redraw correctness and synchronisation
Rendering coordination	helix-editor/helix:helix-event/src/renderer	n-heavy	"global event bus (per runtime) for registering + dispatching hooks", "central coordination of event registration + dispatch across modules/tasks", "write-light (register_event/register_hook)", "runtime_local! { static REGISTRY: RwLock<Registry> = RwLock::new(Registry)", "many reads for dispatching events, few writes for registering events/hooks"	concurrency model	"Arc<Self> enables concurrent transport tasks", "pending_requests: Mutex<HashMap<u64, Sender<Result<Response>>>>", "for each request, a channel is stored so that its response can be delivered later", "match server responses to originating client request", "send task inserts pending requests, recv task removes/completes them"	RuntimeLocal<RwLock<Registry>	ordering but rendering can stall while long-running
Event dispatch	helix-editor/helix:helix-event/src/renderer	n-heavy	"Coordinate editor redraw timing and prevent rendering during async work", "lets background tasks block frame rendering until computation completes, while also waking renderer efficiently", "RENDER_LOCK: RwLock<()> -> controls when rendering is allowed", "async tasks take the read lock, renderer takes write lock, rendering must wait until all async tasks have completed"	Actor / message-	"global event bus (per runtime) for registering + dispatching hooks", "Registry (events map + handlers map)", "runtime_local! { static REGISTRY: RwLock<Registry> }",		async readers hold the lock
		Coordination	"global event bus (per runtime) for registering + dispatching hooks", "central coordination of event registration + dispatch across modules/tasks", "write-light (register_event/register_hook)", "runtime_local! { static REGISTRY: RwLock<Registry> = RwLock::new(Registry)", "many reads for dispatching events, few writes for registering events/hooks"	passing model	"dynamic dispatch via erased hooks"	>, RwLock	Central registry simplifies extensibility and dispatch coordination but shared handler maps require locking and manual type-erasure safety management

		Coordination	"To prevent parallel integration tests from sharing globals while still allowing multi-threaded Tokio runtimes to share state", "Global state isolation for parallel Tokio integration tests", "acquire read lock -> check runtime entry -> upgrade to write path on miss -> insert leaked singleton", "data: RwLock<HashMap<runtime::id, &'static T>> -> rwlock synchronises concurrent access"	Shared-state	"data: RwLock<HashMap<runtime::id, &'static T>>", "read lock -> check runtime entry", "parking_lot::RwLock"	Runtime isolation prevents cross-runtime	
Runtime coordination	helix-editor/helix:helix-event/src/runtime.rs	n-heavy		concurrency model	"write lock", "Multiple threads may belong to same Tokio runtime", "Box::leak is used to make the data live forever"	interference but lazy initialisation paths require additional lock coordination and indirection	
		Coordination	"Instead of locks it uses event-driven concurrency over multiple streams", "Main Application loop of Helix text editor. Responsible for starting the editor, handling input + background jobs, rendering the UI, coordinating everything in a single async event loop", "event-driven async coordination (tokio select)", "Jobs are done through channels", "single main async runtime (tokio runtime), multiple async tasks spawned (UI, jobs, LSP, signals)"	Async executor-driven	"tokio::select!", "single main async runtime (tokio runtime)", "multiple async tasks spawned (UI, jobs, LSP, signals)", "event-driven async coordination", "message passing via channels"	Arc, ArcSwap	Avoids extensive shared-memory locking through async coordination and snapshot sharing, but correctness depends on event ordering and message flow
Execution loop	helix-editor/helix:helix-term/src/app.rs	n-heavy		coordination	"ArcSwap", "Arc<Map>", "Lock-free snapshot sharing", "Immutable transactions", "apply transaction once -> incrementally remap all dependent structures", "some atomic	ArcSwap, Arc,	Snapshot-based sharing gives scalable concurrent reads and async consistency but incremental remapping and transactional coordination logic
		Mixed	"Transactional, incrementally updated shared editor state", "Coordinates text, selections, syntax trees, diagnostics, LSP state, rendering metadata, diff state, undo history, and configuration across multiple editor subsystems and views", "Mostly read-heavy with some local incremental mutation", "Lock-free snapshot sharing, reference-counted ownership, weak lifetime tracking, localised mutex protection (Mutex<Transaction>), event-driven updates", "incremental transactional transforms"	Snapshot / copy-on-write			
State synchronisation	helix-editor/helix:helix-view/src/document.rs			write model	replace (ArcSwap store)"	parking_lot::Mutex, Weak Arc, RwLock, Mutex	become highly complex
		Coordination	"Central coordination layer connecting networking, runtime state, and session lifecycle.", "Arc<RwLock<LoginConfigHandler>> (core session state hub)", "shared mutable coordination across concurrent tasks", "UI thread + Tokio	Shared-state	"Arc<RwLock<LoginConfigHandler>> (core session state hub). lc.read().unwrap() / lc.write().unwrap() across auth + connection", "ENIGO: Arc<Mutex<enigo::Enigo>> -> synchronized shared access to global input controller.", "static ref CLIPBOARD_STATE: Arc<Mutex<ClipboardState>> -> shared lifecycle state for clipboard synchronization thread.", "udp_port = Arc::new(Mutex::new(0)) together with tokio::spawn and udp.lock().unwrap() -> shared mutable coordination between concurrent async tasks during UDP probing."		Centralised shared session state simplifies coordination across networking, IPC, and UI subsystems, but correctness depends on disciplined lock usage
Session coordination	rustdesk/rustdesk:src/client.rs	n-heavy	async tasks + OS worker threads + IPC threads"	concurrency model		Arc, RwLock, Atomic	
		Mixed	"Async layer for desktop session management with networking, video streaming, clipboard synchronisation, transfer jobs, adaptive FPS control, recording, permission propagation, and UI coordination", "Shared video frame queues, decoder FPS telemetry, frame counters, discard/shutdown flags, runtime config (handler.lc), recording state, clipboard ownership/session state, transfer-job state, peer capabilities, permission flags, adaptive-control counters, display worker registry", "Combines message passing for the transfer of ownership with shared-memory for control synchronisation, eventual consistency coordination", "Tokio async network task, decode/render worker threads, render callbacks, clipboard, UI thread, transfer, peer protocol handlers, platform integration callbacks"	Producer-consumer /	"Producer/consumer for queueing", "Channels are used for media/event ownership and shared memory for state and telemetry.", "Arc<RwLock<ArrayQueue<VideoFrame>> -> queue itself is lock free, outer lock serialises queue access data.", "Tokio async network task, decode/render worker threads, render callbacks, clipboard, UI thread, transfer,		Separates ownership transfer from shared runtime telemetry, improving streaming throughput, but coordination between async tasks and worker threads is difficult to reason about
I/O coordination	rustdesk/rustdesk:src/client/io_loop.rs			model	peer protocol handlers, platform integration callbacks"		
		Coordination	"Distributed coordination. Shared memory is used for correctness (dedup, identity consistency, failover decisions), not performance optimization.", "cross-task coordination between async loops, OS threads, and network handler", "prevents duplicate NAT/relay events", "restart/shutdown coordinated via atomic flags"	Shared-state	"SHOULD_EXIT.load(Ordering::SeqCst) loop termination", "SOLVING_PK_MISMATCH.lock().await single resolver", "Arc<RwLock<CONNECT_TIMEOUT>>", "Lock-based shared memory + atomic global flags + event deduplication via shared timestamps"	AtomicBool, Mutex, RwLock, Arc	Shared coordination state enables robust NAT traversal and failover correctness, but distributed lifecycle management depends heavily on correct atomic ordering and locking discipline
Connection coordination	rustdesk/rustdesk:src/rendezvous.rs	n-heavy		concurrency model		AtomicBool, Mutex, RwLock, Arc	
		Coordination	"Shared-state registry + actor-like services + atomic control plane", "multiple async tasks + threads need access", "service lifecycle", "pub-sub shared memory", "AtomicBool for control signals in loops"	Actor / message-	"Services are like actors subscribing to connection events.", "services map uses pub-sub shared memory.", "Shared-state registry + actor-like services + atomic control plane."	Arc, RwLock, Mutex, AtomicBool, Weak Arc	Daemon acts as central authority with shared mutable runtime, separation between immutable config and synchronised mutable subsystems, state that is performance sensitive uses atomics instead of locks, module metadata uses write-once/read-once
Runtime coordination	rustdesk/rustdesk:src/server.rs	n-heavy		passing model	shared memory.", "Shared-state registry + actor-like services + atomic control plane."		
		Coordination	"Multiple worker threads run concurrently -> need synchronised access to global daemon resources and cached state without copying or rebuilding context per request", "pub sql_connection: Mutex<Option<Sqlite3>>, pub manager_info: Mutex<ManagerInfo>, pub boot_stage_lock: Mutex<BootState>, pub zygisk: Mutex<ZygiskState>", "pub zygisk_enabled: AtomicBool -> just a lock = frequent reads, doesn't need locking -> avoid locking overhead", "pub module_list: OnceLock<Vec<ModuleInfo>> -> no mutation after initialisation, only reads after", "all worker threads share one daemon object"	Shared-state	"pub static MAGISKD: OnceLock<MagiskD> = OnceLock::new() -> daemon instance is shared across the whole process, lock -> everyone gets immutable shared access.", "pub sql_connection: Mutex<Option<Sqlite3>>, pub manager_info: Mutex<ManagerInfo>, pub boot_stage_lock: Mutex<BootState>, pub module_list: OnceLock<Vec<ModuleInfo>>, pub zygisk_enabled: AtomicBool, pub zygisk: Mutex<ZygiskState>, pub cached_su_info: AtomicArc<SuInfo> -> shared state for MagiskD, many threads can mutate zygisk/manager info/sql connection/boot stage lock -> only one thread can access it at a time", "all worker threads share one daemon object.", "concurrent threadpool + global daemon -> need synchronisation"	OnceLock, Mutex, AtomicBool, AtomicArc	
Runtime coordination	topjohnwu/Magisk:native/src/core/daemon.rs	n-heavy		concurrency model	"multi-producer + single-consumer architecture: many threads call magisk_log_to_pipe and new_daemon_thread creates logging thread.", "Many producer threads log concurrently, dedicated daemon thread consumes FIFO and serialises logfile output", "shared FIFO pipe connection to logging daemon", "It's guaranteed to always write the whole thing atomically -> kernel synchronisation trick to ensure atomicity, avoids mutex, multiple threads can write concurrently."	AtomicArc, Mutex, AtomicI32, Arc	High-performance logging design: synchronisation protects handle ownership, not IO itself, it uses Arc to avoid serialised writes, uses POSIX PIPE_BUF guarantees instead of userspace locking, separation between daemon logging path and zygisk path
		I/O-bound	dedicated logging daemon thread", "Global logging pipe FD, zygisk raw log FD, shared FIFO pipe connection to logging daemon", "many producer threads log concurrently, dedicated daemon thread consumes FIFO and serialises logfile output"	Producer-consumer /			Fast path: lock-free cache reuse, slow path: rebuild policy from DB -> atomically publish shared object. Locking around mutable authorisation state -> small contention. Arc allows request handlers to retain policy snapshots.
Logging system	topjohnwu/Magisk:native/src/core/daemon.rs			model		AtomicArc, Mutex, Arc	
		Coordination	"Cache + synchronizs root access policy + state across concurrent su requests in the daemon", "Multiple daemon worker threads spawned from ThreadPool::exec_task, handle Unix socket SUPERUSER requests", "pub cached_su_info: AtomicArc<SuInfo> -> shared cache, every worker thread handling an incoming su request can access it concurrently", "Mutex<AccessInfo> -> mutable synchronised data: only the access policy state is mutable"	Shared-memory	"lock-free + mutex synchronisation, atomic pointer swaps for global cache visibility,		
Caching layer	topjohnwu/Magisk:native/src/core/daemon.rs	n-heavy		concurrency model	mutex for interior mutable state"		

		Coordination	"Shared task dispatch and worker lifecycle coordination for daemon thread pool", "Multiple producer threads need to submit work to reusable worker threads while coordinating worker sleep/wakeup and dynamic thread creation", "Producers lock pool state, wait while task slot occupied, insert task, then either wake idle worker or spawn new thread", "Workers lock state, advertise idleness, sleep on condvar, take task, release lock"	Worker-thread pool	"Shared task dispatch and worker lifecycle coordination for daemon thread pool", "Multiple producer threads need to submit work to reusable worker threads while coordinating worker sleep/wakeup and dynamic thread creation", "Producer threads enqueue work concurrently, worker daemon threads compete for shared task slot, core threads persist while extra threads self-terminate after idle timeout", "static THREAD_POOL: LazyLock<ThreadPool> = LazyLock::new(ThreadPool::default) -> single shared thread pool, initialised once and shared across threads"	Mutex, Condvar, LazyLock	Only one shared task slot exists, so task submission is serialised, synchronisation separates task execution from lock ownership, condition variables
Resource coordination	topjohnwu/Magisk:native/src/core/	n-heavy I/O-bound	"WebDAV upload / download operations, coordinates sb / settings",	execution model Producer-consumer / pipeline coordination	"Multiple components must trigger sync while avoiding duplicate uploads and recursive sync loops", "Concurrent producers send change signals, async worker consumes and batches events", "Actor/worker-style architecture, channels, bounded channel acts as flag queue, debounce merges bursts of events", "static DB_CHANGE_TX: OnceLock<Sender<String>> = OnceLock::new(); -> one sender is shared across the whole application, any component can notify the db has changed"	arc, OnceLock, AtomicUsize	coordinate state transitions Debouncing and batching reduce duplicate sync
State synchronisation	farion1231/cc-switch:src-tauri/c	I/O-bound	"run_with_webdav_lock(...) await -> only one operation can run at once" "Coordinate background WebDAV auto-sync across async tasks", "async	model Shared-state	"Require shared access to the same application resources throughout runtime", "Many reader/few writer", "Concurrent reads from background workers with some synchronised mutations", "Services accessible through AppState, cloned via Arc, coordinated through async primitives", "concurrent tasks can own the database, no copying of db", "multiple threads can read auth state concurrently + mutate config/tokens", "app.manage(CopilotAuthState(Arc::new(RwLock::new(copilot_auth_manager))))", "background tasks clone the db w/o copying the resources", "let db_for_timer = state.db.clone(); -> background tasks clone the db w/o copying the resources."	arc, rwlock	make behaviour harder to reason about Centralised shared services simplify cross-
Runtime coordination	farion1231/cc-switch:src-tauri/li		worker consumes and batches events"	concurrency model	"Multiple async tasks must view and update one provider health view so routing logic can reject unhealthy providers", "Shared-memory async coordination: lock-free atomic coordination for counters and permit management, async reader/writer locking for coordinated state transitions and configuration mutation", "Many tasks can share the same breaker, and all mutations need to be thread-safe.", "CircuitBreaker: state: Arc<RwLock<CircuitState>>", "Arc is used throughout the structure to allow multiple tasks to use the breaker simultaneously"	arc, rwlock, AtomicU32	component access, but global state increases coupling and requires careful async lock management
I/O coordination layer	farion1231/cc-switch:src-tauri/src/p	n-heavy Mixed	"Arc<RwLock<CircuitState>>.consecutive_failures: Arc<AtomicU32>" "Handling of streaming/non-streaming responses, Sse usage, timeout"	concurrency model Async executor-driven	"Async coordination under Tokio runtime.", "runtime task scheduling (tokio::spawn), stream polling via StreamExt::next(), timeout with tokio::time::timeout.", "Tokio async runtime. Concurrency is task-based.", "tokio::spawn schedules logging onto runtime worker threads.", "tokio::sync::Mutex is chosen because locks may be held across .await points.", "Request handlers collect data, then detached tasks persist it asynchronously."	arc, mutex, rwlock (tests), AtomicBool	locking and careful visibility guarantees Streaming responses remain responsive because
Request coordination	farion1231/cc-switch:src-tauri/src/p	I/O-bound	arrive through the stream loop and mutate local parsing buffers" "HTTP proxy server handling API requests, routing, failover, circuit breaker",	coordination Shared-state	"Maintain consistent routing across requests, allow runtime config updates, expose shared server status, coordinate provider health globally", "async tasks clone shared Arc references and access shared state via async locks", "Shared-memory coordination with async task scheduling", "they all share the same db, config, status, start time, provider status, failover manager", "circuit breaker state persists across requests"	arc, rwlock	Shared long-lived routing and failover state improves consistency across requests, but increases
Resource coordination	farion1231/cc-switch:src-tauri/src/p		"task per connection"	concurrency model	"Prevent concurrent sync corruption, coordinate database imports/exports, avoid overlapping manual/automatic sync", "shared-state coordination using async mutual exclusion", "many async tasks may exist concurrently but only one sync operation executes at a time", "run_with_webdav_lock(...) await -> only one operation can run at once", "webdav_sync_mutex().lock().await", "it needs to prevent db corruption or inconsistent snapshots"	tokio::sync::Mutex, Arc	coupling between subsystems Single async mutex guarantees consistency for sync
Event dispatch system	farion1231/cc-switch:src-tauri/src/s	n-heavy	avoid overlapping manual/automatic sync"	model	"Multiple UI components need synchronised access to the same editor and configuration state without duplicating data", "Reactive reads/writes using .get(), .with(), .update(), .get_untracked()" and "Uses reactive programming and shared ownership rather than explicit thread synchronisation" and "children stores size, layout rectangle signal and child content, so the layout information is shared"	Rc, RwSignal	operations, but serialises uploads/downloads and may delay concurrent requests Reactive snapshot updates minimise recomputation
Rendering coordinat	lapce/lapce:lapce-app/src/editor.rs	n-heavy	"Tracks installed/available plugins + installs/uninstalls them", "background	concurrency model Producer-consumer / pipeline coordination	"Reactive shared-state", "Single-threaded UI/reactive runtime context", "RwSignal is reactive mutable state, so shared variable + automatic UI updates" "worker threads return results via callbacks", "background threads fetch plugins: std::thread::spawn(move { let volts = Self::query_volts(&query, offset); send(volts); });" "volts.update(volts { volts.extend(...) }); -> volts to shared UI state", "UI components and worker tasks must observe/update shared plugin state", "many std::thread::spawn"	RwSignal, Arc, AtomicU64	restricted to single-threaded UI execution Asynchronous plugin coordination reduces blocking during installs and updates but maintaining consistent UI state requires careful lifecycle and ordering management
State synchronisation	lapce/lapce:lapce-app/src/plugin.rs	Coordination	threads fetch plugins" "dispatcher for the editor, receives requests+notifications from UI, routes,	model Shared-state	"workspace_fs_change_handler: Arc<Mutex<Option<Sender<bool>>>>", "let mut handler = self.workspace_fs_change_handler.lock(); -> lock mutex", "Arc allows multiple threads to own the data, mutex so that one thread accesses it at the time", "all can read/replace/clear handler", "debounce worker thread", "batch rapid file events into a single delayed update cycle"	Arc, Mutex, AtomicU64	Pipeline-style batching and event routing improve responsiveness but coordinating many independent
Event dispatch system	lapce/lapce:lapce-proxy/src/dispatch	n-heavy I/O-bound	filesystem changes", "Arc<Mutex<Option<Sender<bool>>>>" "LSP communication via RPC and document sync state" and	concurrency model Actor / message-	"channels are used for coordination", "most data transferred via channels and ownership moves", "stdin writer thread, stdout reader thread, stderr logging thread" "Primarily message-passing, some shared-memory synchronisation", "server_rpc.clone() and plugin_rpc.core_rpc.clone() might use shared-mem internally", "low shared-memory usage"	Arc, Mutex	event producers increases orchestration complexity Channel-driven coordination minimises shared mutable state but correctness depends strongly on RPC ordering and synchronisation between streams
Connection coordinat	lapce/lapce:lapce-proxy/src/plugin/		"stdin/stdout/stderr threads coordinate primarily through RPC channels"	passing model			

		Coordination	"RPC request/response matching via pending map", "Central RPC mailbox,	Actor / message-	"tx: Sender<CoreRpc>, rx: Receiver<CoreRpc>", "message-passing channels", "when a request is sent, the caller creates response channel", "pending shared: table of pending requests waiting for responses", "channels carry ownership of large payloads while shared memory is limited to lightweight coordination metadata", "minimal shared-memory usage and minimal locking"	Arc, Mutex, AtomicU64	Mailbox-style coordination keeps most request flow asynchronous but the shared pending-request map can still become a central synchronisation
Request coordination	lapce/lapce:lapce-rpc/src/core.rs	n-heavy I/O-bound	event-loop architecture with lightweight shared metadata" "coordinate HTTP client pool + async request cancellation/state sharing",	passing model Resource arbitration	"static GLOBAL_CLIENT: RwLock<Option<Arc<Agent>>> = RwLock::new(None)", "reuse	Arc,RwLock	bottleneck Shared pooled HTTP clients reduce TCP/TLS setup overhead, but the global RwLock introduces coordination overhead during initialization and reset paths
Resource coordination	nushell/nushell:crates/nu-command	Mixed	"GLOBAL_CLIENT: RwLock<Option<Arc<Agent>>>" "streamwriter produces stream sata, streamreader consumes it,	model Actor / message-	tcp connections + tls handshake", "many threads can own the same client safely" "Streamreader receives messages", "uses messagepassing for receiver",	Arc,Mutex,Condvar	ACK-based backpressure improves flow control and prevents overload, but lifecycle coordination between writers, readers, and manager state is complex
Request coordination	nushell/nushell:crates/nu-plugin-co		streammanager tracks active streams", "uses message passing for receiver"	passing model	"StreamWriterSignal: Condvar -> allows efficient thread sleeps"		Layered coordination combines shared immutable metadata with mutex-protected runtime services, but the large central runtime state increases coupling and synchronisation complexity
		Mixed	"runtime state, engine state stored parsed files and modules, commands, declarations", "pub repl_state: Arc<Mutex<ReplState>>, pub jobs:	Shared-state	"pub repl_state: Arc<Mutex<ReplState>>, pub jobs: Arc<Mutex<Jobs>>, pub	Arc,Mutex,AtomicBool,	Simple shared registry design guarantees correctness during handler execution, but the mutex serialises registration and dispatch operations
Runtime coordination	nushell/nushell:crates/nu-protocol/		Arc<Mutex<Jobs>>" "Threads register handlers. Event handlers", "Arc<Mutex<Vec<usize,	concurrency model Shared-state	regex_cache: Arc<Mutex<LruCache<String, Regex>>>" "coordinates shared-memory. Threads register handlers. Event handlers" and "Multiple threads need to register, remove, execute handlers through a shared registry" and	AtomicU32, Arc::make_mut arc, Send/Sync	correctness during handler execution, but the mutex serialises registration and dispatch operations
Event dispatch system	nushell/nushell:crates/nu-protocol/	n-heavy	access"	concurrency model	locked during handler execution, favouring correctness over concurrency"		Read-heavy cache scales efficiently through RwLock and per-key OnceLock initialisation, but insertion and initialization paths remain coordination-heavy
Caching layer	nushell/nushell:crates/nu-utils/s	CPU-bound	"map of keys + lazily initialised values", "OnceLock means that the value starts out as uninitialised, then the first thread initialises it"	Initialisation / singleton coordination	"Box<OnceLock<V>>>", "OnceLock serialises initialisation per key", "LazyLock so that it delays the creation of the map until first use"	RwLock, OnceLock, Box, LazyLock	Immutable shared configuration minimises synchronisation overhead, but this layer only bootstraps downstream threadpool coordination
Runtime coordination	sharkdp/fd:src/main.rs		command.map(Arc::new)" "parallel filesystem scanning, filter entries and send matching results between	model Producer-consumer /	command.map(Arc::new) wraps the command in Arc before storing it in Config" "worker threads put WorkerResult into batches" and "receiver threads consume	Arc,Mutex,AtomicBool	Batching and bounded queues improve filesystem traversal throughput, but cancellation flags and batch ownership coordination add implementation complexity
I/O coordination layer	sharkdp/fd:src/walk.rs		threads", "worker threads put WorkerResult into batches ... send the batches through channels"	pipeline coordination model	batches", "communication between threads uses channels crossbeam_channel::bounded"		Uses snapshotting pattern, minimises lock duration, avoids holding locks during UI updates, relies on Arc
Runtime coordination	sxyazi/yazi:yazi-core/src/tasks/tasks	Coordination	"Async task monitoring and UI progress synchronisation around a shared scheduler", "Multiple async tasks need concurrent access to task progress", "Periodic polling loop acquires locks, derives snapshots (TaskSummary, TaskSnap), releases lock immediately, then performs UI updates", "Scheduler object, ongoing task collection", "Shared state coordination via lock-protected data, cooperative async task lifecycle management", "lock only to copy state, then drop before AppProxy::update_progress(new)"	Shared-memory Shared-state	"Async task monitoring and UI progress synchronisation around a shared scheduler" and Arc "arc scheduler object, ongoing task collection" and "One background Tokio task periodically monitors shared scheduler state while UI also read shared state" and "lock shared task state, compute summary, compare with prev summary" and "snapshotting pattern, minimises lock duration"	Arc	for shared ownership instead of channels
Event dispatch system	sxyazi/yazi:yazi-dds/src/pubsub.rs	n-heavy	"pubsub event routing for lua plugin, dispatches events to multiple targets, remote peers", "maps event-kind to plugin to callback, shared PEERS/BOOT state for routing decisions", "Shared-state routing + pub/sub, decision driven dispatch across execution and network", "Read-heavy LOCAL.read().contains_key(n), REMOTE.read().keys().cloned().collect(), writes only happen in sub/unsub", "Shared memory is used only for subscription + routing decisions, execution and transport are w/o locks"	Shared-state concurrency model	"pub static LOCAL: RoCell<RwLock<HashMap<,>, "LOCAL.read().contains_key(n)", "writes only happen in sub/unsub"	RwLock	Registry as routing table, snapshot, no long held
Connection coordination	sxyazi/yazi:yazi-dds/src/server.rs	I/O-bound	TCP server with multiple clients with message routing, broadcast updates, and async I/O handling", "each task reads from socket, writes via channel"	Actor / message- passing model	"broadcast via cloned channel senders", "(tx, mut rx) = mpsc::unbounded_channel::<String>()", "Client { id, tx: UnboundedSender<String> }"	RwLock	locks, event dispatch pipelines Shared-memory: directory, message-passing: communication, use of snapshot, lock minimised / never held across async IO, broadcast via cloned channel senders
Caching layer	sxyazi/yazi:yazi-runner/src/loader/lo		plugin loader, cache for lua module. Load plugin files from disk, caches in memory, compile into tables", "To avoid repeated disk reads, repeated Lua parsing/compilation", "Shared global plugin cache mapping plugin id -> Chunk", "rare write, in-place mutation under write lock"	singleton coordination model	"pub static LOADER: RoCell<Loaders>", "cache: RwLock<HashMap<String, Chunk>>>", "lazy binary compilation", "package.loaded registry"		caching (Rust cache + Lua package.loaded), lazy binary compilation, optimised for many readers and rare writers
Runtime coordination	sxyazi/yazi:yazi-scheduler/src/sche	Coordination	"scheduler: creates tasks, assigns ids, routes them into worker queues. Dispatch", "reuse across multiple tasks, cross thread coordination, deduplication of work, shared control state across async tasks without copying", "lock registry, create task, release lock, dispatch to worker queue", "self.preload.loading.lock().put(hash, id)", "self.ongoing.lock().cancel(id)"	Producer-consumer pipeline model	"Worker + channels, shared-state coordination, event completion (CompletionToken)", Arc "self.file.submit(#in, LOW)", "self.hook.submit(hook, HIGH)", "self.preload.loading.lock().put(hash, id)", "Throttle::new(...)"	Arc	Arc is for system level coordination, most data flow is message-passing, Arc is used for shared engines and registries, ID avoids direct shared mutable structs
		n-heavy	"self.preload.loading.lock().put(hash, id)", "self.ongoing.lock().cancel(id)"	pipeline model	"self.preload.loading.lock().put(hash, id)", "Throttle::new(...)"		Separation between execution and coordination: work is message-passed, lifecycle/cancellation/progress are shared memory, avoids holding locks across .await, subsystem sharing
Resource coordination	sxyazi/yazi:yazi-scheduler/src/work	Mixed	"Async worker runtime: task execution, cancellation, lifecycle tracking", "ongoing: Arc<Mutex<Ongoing>>"	Worker-thread pool execution model	"pub struct Worker", "Arc<Mutex<Ongoing>>>," "async_priority_channel::unbounded()", "tokio::spawn(async move {", "(0..YAZI.tasks.file_workers.get())"	Arc,parking_lot::Mutex	

Request coordination		Coordination	"API thread sends VmmAction through channel -> writes EventFd, VMM thread wakes via epoll/EventManager, receives request, locks Vmm briefly to execute action, sends response back through reverse channel", "let (to_vmm, from_api) = channel(); let (to_api, from_vmm) = channel();", "Message passing over shared-state", "pause mode blocks the EventManager and processes only API commands until resume"	Actor / message-passing model	"API thread sends VmmAction through channel -> writes EventFd", "let (to_vmm, from_api) = channel();", "sends response back through reverse channel"	arc,mutex	API requests are isolated from the VM and sent through message passing to the VMM thread, which keeps VM state safe, but adds overhead and delays because all changes must go through a queue and event system.
layer	firecracker-microvm/firecracker/src	n-heavy					
Runtime		Coordination	"launches threads, and enters the event loop", "Coordinate VM state, metrics, device events, and communication between event loop and VCPU threads", "Main thread runs EventManager, VCPU threads execute guest CPUs, optional API threads interact with VMM", "event_manager.add_subscriber(firecracker_metrics.clone()); event_manager.run();"	Actor / message-passing model	"Main thread runs EventManager", "event_manager.add_subscriber()", "communication between event loop and VCPU threads"	arc,mutex	The main process coordinates VM, metrics, and threads using shared state and event loops, which keeps structure clear, but increases coupling and requires synchronisation across many components
coordination layer	firecracker-microvm/firecracker/src	n-heavy					
Runtime		Coordination	"Assemble the VM, wire together shared state, launch the execution threads", "All components operate on the same virtual machine", "An Arc reference of the built Vmm is also plugged in the EventManager, while another is returned", "multiple threads/components can own references to the same VM object", "// The device mutex mustn't be locked here otherwise it will deadlock."	Initialisation / singleton coordination	"Assemble the VM, wire together shared state, launch the execution threads", "let vm = Arc::new(vm); let vmm = Arc::new(Mutex::new(vmm));", "An Arc reference of the built Vmm is also plugged in the EventManager", "multiple threads/components can own references to the same VM object"	arc,mutex	VM setup shares state across components using Arc and Mutex so everything can access the same VM, but this increases coupling between initialisation and runtime systems
coordination layer	firecracker-microvm/firecracker/src	n-heavy					
Runtime		Mixed	Owens the Vmm struct", "device manager", "vCPU threads", "snapshotting",	Actor / message-passing model	"Each vCPU runs in its own OS thread", "Main VMM thread (pub struct Vmm) -> API commands", "EventManager thread -> event loop, dispatches I/O + device + vCPU events", "communicates back via channels"	arc,mutex,barrier	Event-driven design with threads for vCPUs and devices, which improves performance by separating execution and control, but requires complex coordination using channels, barriers, and event signals
coordination layer	firecracker-microvm/firecracker/src	Mixed	"device updates", "guest memory mappings"	passing model	events", "communicates back via channels"		
OS integration bridge			"Implements vCPU lifecycle, thread execution", "self.run(filter)", "KVM execution loops", "Receiver<VcpuEvent>", "Sender<VcpuResponse>",	Worker-thread pool	"thread per vCPU", "thread::spawn(move { self.run(filter); });", "each vCPU runs in its own thread and communicates through message passing, which enables parallel execution, but adds complexity for synchronisation and coordination between threads"	arc,barrier	Each vCPU runs in its own thread and communicates through message passing, which enables parallel execution, but adds complexity for synchronisation and coordination between threads
Resource	firecracker-microvm/firecracker/src	Mixed	"barrier.wait()"	execution model	own OS thread", "startup barrier synchronisation"		
			"Mutex<HashMap>", "Mutex<ResourceAllocator>", "AtomicU32", "guest memory", "KVM VM object"	Shared-state	"Multiple vCPU threads and device handlers must access the same VM resources", "Guest RAM (GuestMemoryMmap)", "interrupt routing table (Mutex<HashMap>)",	arc,mutex,AtomicU32	Shared VM state is accessed by many threads using locks and atomics, which keeps memory consistent, but introduces performance overhead and increases complexity
coordination layer	firecracker-microvm/firecracker/src		"memory region registration", "snapshotting support",				
Shared configuration		CPU-bound	"GuestMemoryMmap", "register_memory_region"	concurrency model	"resource allocator (Mutex<ResourceAllocator>)"		
			"Take the parsed ASTs and verify if everything is allowed before execution", "expression.references()", "Analyzer::resolve_call(functions, name, arguments)?,"	Initialisation / singleton coordination	"Take the parsed ASTs and verify if everything is allowed before execution", "Read only access", "recipes are constructed once and then treated as read-only"	arc	Static analysis over immutable shared data makes execution simple and fast, but it assumes everything never changes during analysis
system	casey/just/src/analyzer.rs			model			
Shared configuration		CPU-bound	"Represent a recipe dependency with arguments + support serialisation and formatting", "Mostly reads (recipe.name(), serialisation traversal, display traversal)",	Initialisation / singleton coordination	"Represent a recipe dependency with arguments + support serialisation and display formatting", "Mostly reads (recipe.name(), serialisation traversal, display traversal)",	arc	Very lightweight representation of dependencies keeps the system simple and fast, but only supports static traversal and formatting
system	casey/just/src/dependency.rs			model			
Runtime		Mixed	display formatting", "recipe.name()", "serialisation traversal, display traversal"	Resource arbitration	"Arc for ownership/lifetime management instead of coordination"	Arc, mutex	Parallel execution of dependencies improves throughput, but requires locking to prevent duplicate execution, which adds overhead
			"Dependency execution, prevents duplicate runs when run in parallel", "Allow concurrent execution of dependency recipes + ensure a recipe runs at most once", "let mutex = ran.mutex(recipe, arguments);", "Dependency worker threads spawned in run_dependencies()"		"prevent duplicate runs when run in parallel", "Dependency execution", "let mutex = ran.mutex(recipe, arguments); let mut guard = mutex.lock().unwrap();", "prevent the same recipe from executing twice, coordinate the execution across threads"		
coordination layer	casey/just/src/justfile.rs			model			
State synchronisation		Coordination	"Track whether a recipe path + arguments has already executed and prevent duplicate execution across parallel dependency threads", "Multiple threads may reach the same dependency at the same time and must coordinate so it runs only once", "entry(recipe.recipe_path().clone())"	Resource arbitration	"Track whether a recipe path + arguments has already executed and prevent duplicate execution across parallel dependency threads", "Multiple threads may reach the same dependency at the same time and must coordinate so it runs only once", "Arc<Mutex<bool>> is shared and locked by workers before execution"	arc,mutex	Central registry prevents the same work from running twice, which ensures correctness in parallel execution, but the global lock can become a bottleneck when many tasks are started at once
layer	casey/just/src/ran.rs	n-heavy		model			
OS integration bridge		Coordination	"Install OS signal handlers, track child processes and forward signals to child processes when needed", "static INSTANCE: Mutex<SignalHandler> = Mutex::new(SignalHandler::new());", "children: BTreeMap<32, Command>", "handle() through registered signal callbacks"	Shared-state	"Install OS signal handlers, track child processes and forward signals to child processes when needed", "Exclusive mutable access through mutex", "static INSTANCE: Mutex<SignalHandler>"	mutex	Signal handler keeps system-wide behavior consistent, but all signal updates go through a single lock, which limits parallel handling
	casey/just/src/signal_handler.rs	n-heavy		concurrency model			